

Nesne Yönelimli Analiz ve Tasarım

Nesnelerin Belirlenmesi

Sınıf Şeması

Nesneler Arası Bağlantılar

Modelin Koda Dönüştürülmesi

Örnek Uygulamalar (Veri Topluluklarıyla İşlemler),
Örnekler (Sıralama Şeması, Etkinlik Şeması, Durum Şeması)

Nesne Yönelimli Analiz

- * Analiz aşamasının ilk adımı gereksinimlerin belirlenmesidir.
- * Gereksinimler, sistemin düzgün çalışması için yapması beklenenleri ifade eder
- * Gereksinimler ilk olarak kullanıcılarla (paydaşlarla) görüşülerek (gerekli ise standart, yasa, yönetmelik vb. dokümanlar takip edilerek) oluşturulur, ve neler istendiği belirlenir.
- * Daha sonra kullanım durumları (use case) belirlenerek gereksinimler ayrıntılandırılır.
- * Sistemin tüm istenentlerle birlikte, beklenmeyen ya da istenmeyen durumları da göz önüne alınır.
- * Kullanım durumları analiz edilerek nesneler, nesne üyeleri ve nesneler arasındaki bağıntılar belirlenir.

Nesnelerin Belirlenmesi

- * NYP gerçek dünyayı daha doğru modelleyebilmek için geliştirilmiştir.
- * Gerçek hayatta çevremizde gördüğümüz her şey nesnedir.
 - * insan, ürün, sipariş, fatura, öğrenci, ekran, televizyon, masa, sıra, bisiklet, araba, köpek v.s.
- * Nesnelerin **durum** ve **davranışları** bulunur.
 - * İnsan için durumlar; adı, yaşı, boyu v.s. iken davranışlar; öğrenmek, anlamak, uyumak, konuşmak, koşmak v.s.
 - * Bisiklet için durumlar; rengi, o anki vitesi, hızı, tekerlek sayısı, vites sayısı v.s. iken davranışlar; fren yapması, hızlanması, yavaşlaması, vites değiştirmesi v.s.
- * Yazılım nesneleri de durum ve davranışlara sahiptir.
 - * Nesnelerin durumları nitelik (özellik) olarak da adlandırılır ve **üye değişkenler** ile ifade edilir.
 - * Davranışlar ise **üye fonksiyonlar/methodlar/yöntemler** ile temsil edilir.

Nesnelerin Belirlenmesi

- * Gereksinimlerden nesnelerin belirlenmesi işlemi için; nesne modelleme, alan modelleme, kavramsal modelleme gibi isimler kullanılır.
- * Nesneler belirlenirken, analiz aşamasında oluşturulan gereksinim listesi (kullanım durumlarının ayrıntıları) üzerinde metinsel analiz yapılır (Abbott's tekniği).
- * Gereksinim listesindeki **isim ve isim tamlamaları sınıf adaylarıdır.** **Özellikleri** ya da **davranışları** bulunan isim ya da isim tamlamaları sınıf olabilir.
- * Gereksinim listesindeki **eylemler** nesnelerin **üye yöntemlerini** ifade edebilir.
- * Bu işlem sadece bir başlangıçtır ve yinelemeli bir süreçtir.
- * Gereksinim listesi incelendikçe yeni sınıflar, özellikler, yöntemler, sınıflar arası bağlantılar belirlenir. Sınıflar değiştirilebilir, niteliğe dönüşebilir, kullanılmayabilir. Nitelikler sınıfa dönüşebilir vb.
- * Sınıflar belirlendikten sonra tasarım ayrıntıları içermeyen **Analiz seviyesi Sınıf Şeması (kavramsal model)** oluşturulur.

Nesnelerin Belirlenmesi

Nesne adayları (Craig Larman):

- * Fiziksel ya da elle tutulabilir nesneler (ürün, insan, bilgisayar, klavye, ekran ...)
- * Yerler (okul, bina, yerleşke, sınıf, oda ...)
- * İşlemler (transactions) (para çekme..., kayıt yaptıрма...)
- * Roller (yönetici, öğrenci, personel, kayıtlı kullanıcı v.b.)
- * Harici sistemler (Veritabanları, sürücüler, banka bilgi sistemi, web servisleri...)
- * Kuruluşlar (okul, işyeri, firma...)
- * Olaylar (ActionListener, ActionEvent, KeyListener, KeyEvent...)
- * Loglar ...

Nesnelerin doğru olarak belirlenmesi için en etkili yol paydaşlarla görüşmektir (beyin fırtınası).

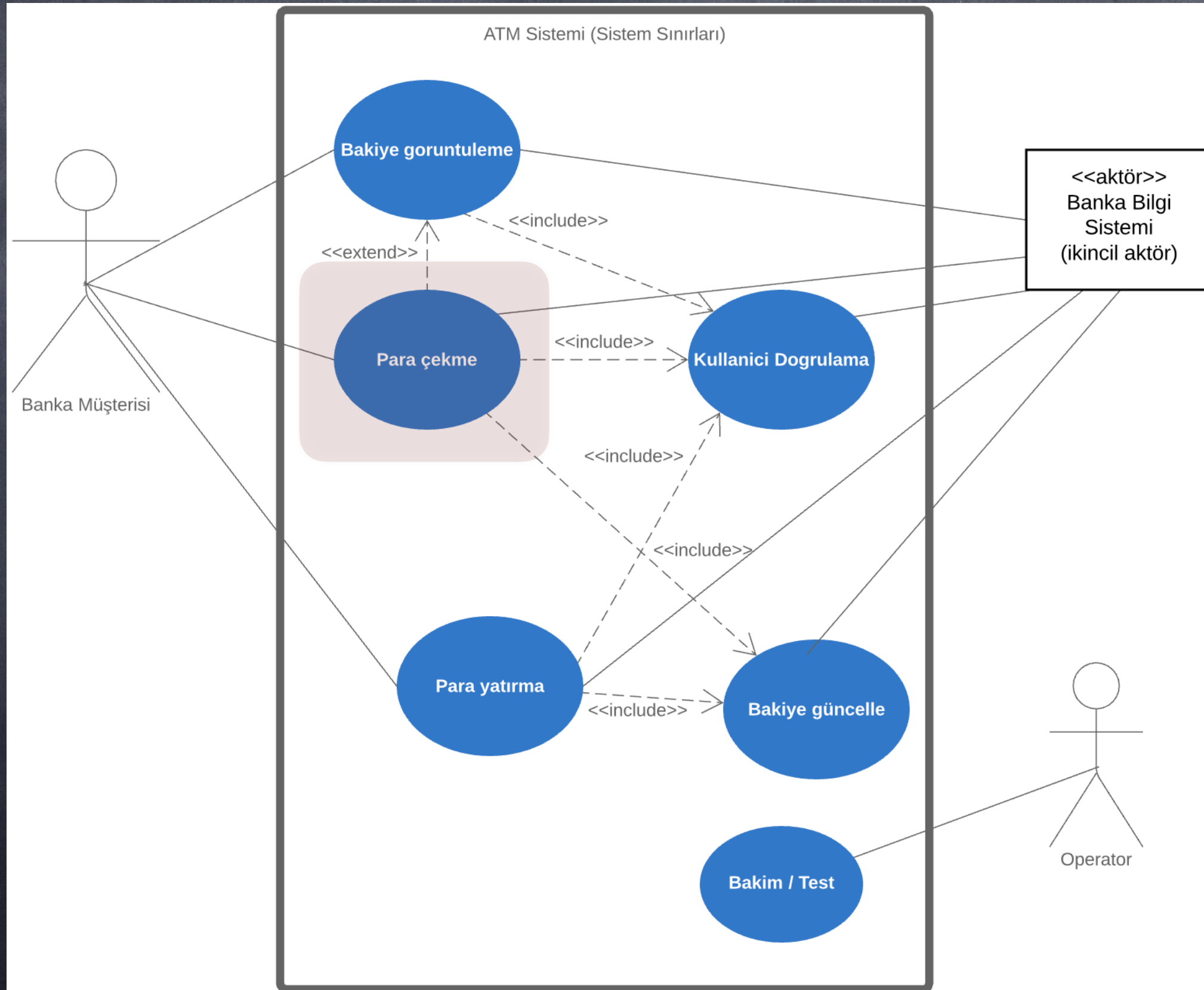
ATM Sistemindeki Nesnelerin Belirlenmesi

- * ATM sistemi aşağıdaki donanım bileşenlerinden oluşur:
 - * Tuş takımı; ile veri girişi yapılır.
 - * Ekran; kullanıcıya mesaj görüntüler.
 - * Para bölmesi; para alma ve para verme işleminden sorumludur.
 - * Kart bölmesi; banka kartını alır ve doğrular.
- * Müşteriler hesaplarından banka bilgi sistemini kullanarak; para çekme, para yatırma, bakiye görüntüleme vb. işlemlerini yapabilmelidirler.
- * Banka görevlileri ATM sisteminin bakımını (para doldurma, sıkışan/ yutulan kartları alma v.b.) yapmalıdır.
- * Tüm işlemlerde güvenlik gereksinimleri göz önünde bulundurulmalıdır (yetki kontrolü).
- * Sistem, Banka Bilgi Sistemiyle gevrimiği çalışmalıdır. Kullanıcılar Banka Bilgi Sisteminde yer alan hesaplarına erişerek işlemlerini gerçekleştirirler.

Not: Sarı renkliler nesne (sınıf), yeşil renkliler yöntem adaylarıdır.
Aynı cümlede bulunan nesneler bağıntılıdır.

ATM Sistemindeki Nesnelerin Belirlenmesi

Para Çekme Kullanım Durumu (Use Case)



Use Case- "Para Çekme" Olay Akışı (Ana senaryo-başarılı)

1. ATM sistemi Ekrana, müşteriden "kartını kart bölmesine takmasını" isteyen mesaj yazdırır.
2. Kart sahibi kartını kart bölmesine takar.
3. Kart bölmesine kart doğrulamasını yapar.
4. Ekrana şifre girilmesini isteyen mesaj gönderilir. kullanıcı doğrulama
5. Tuş takımını kullanarak girilen şifreyi alır.
6. Kullanıcı doğrulama ve yetki kontrolü için banka bilgi sistemine istek gönderilir.
7. Banka bilgi sistemi erişim isteğini kabul eder ve gerekli bilgileri (Müşteri hesabı) gönderir.

8. ATM sistemi ana menüyü gösterir ve yapılacak işlemin seçilmesini ister. işlem seçimi

9. Ekrana, çekilecek tutarlar listelenir (ParaÇekme).

10. Tuş takımı ile çekilecek tutar alınır.

11. Kart bölmesi kartı çıkartır. para çekme işlemi

12. Kart sahibi kartı alır.

13. ATM sistemi parayı ve makbuzu verir.

14. Kart sahibi parayı ve makbuzu alır.

ATM Sistemindeki Nesnelerin Belirlenmesi

"Para Çekme" Kullanım Durumu Olay Akışı (Alternatif akışlar)

A1. Kart doğrulanamadı (3)

4. Kart bölmesi kartı çıkartır.

5. İşlem sonlandırılır.

A2. Yanlış şifre (6)

8. 3 den az kez yanlış ise yeniden gir.

8. 3 kez yanlış girilmiş ise kart yutulur.

9. işlem sonlandırılır.

A3. Yeterli para yok (9)

10. ATM de yeterli bakiye yoksa yeni tutar girmesi istenir.

11. Hesapta yeterli bakiye yoksa yeni tutar girmesi istenir.

A4. Banka kartı alınmadı(10)

11. 30 sn. boyunca alınmadı ise kart yutulur

12. Gerekli bilgilendirme yapılır

13. İşlem sonlandırılır

A5. Para alınmadı (12)

13. 30 sn. boyunca alınmadı ise para yutulur

14. Gönderilen ve yutulan para karşılaştırılır.

15. Gerekli ise hesap güncellenmesi yapılır

12. Gerekli bilgilendirme yapılır

13. İşlem sonlandırılır

...

UML - Sınıf Diyagramı

- * Yapısal gösterim şekillerindendir.
- * Tasarlanan sistemin sınıflar ve arayüzler seviyesinde statik yapısını gösterir.
- * Sınıf ve arayüzlerin; özelliklerini, üyelerini, kısıtlarını ve bağıntılarını içerir.

GeometrikSekil

GeometrikSekil sınıfının basit gösterim

GeometrikSekil

x:double
y:double
renk:String

konumDegistir()
alanHesapla()
toString()

GeometrikSekil sınıfının analiz seviyesi gösterimi

GeometrikSekil

- x:double
- y:double
- renk:String="Beyaz"
+ toplamSekilSayisi: int

+ konumDegistir(double,double):void
+ alanHesapla() : double
+ toString() : String

GeometrikSekil sınıfının gerçekleştirme seviyesi gösterimi

GeometrikSekil

private
x:double
y:double
renk:String="Beyaz"

public
konumDegistir(double,double):void
alanHesapla() : double
toString() : String

GeometrikSekil sınıfının gerçekleştirme seviyesi gösterimi

GeometrikSekil

- x:double
- y:double
- renk:String="Beyaz"
+ toplamSekilSayisi: int

+ konumDegistir(double,double):void
+ alanHesapla()
+ toString()

Soyut (Abstract) Sınıf

<<interface>> Sekil

+ konumDegistir(double,double):void
+ alanHesapla()

Arayüz (Interface)

+ : public
- : private
: protected
altı çizili: statik üye

Modelin Koda Dönüştürülmesi / Sınıf

```
package cc.ders6;
```

```
public class GeometrikSekil {
```

```
    private double x;
```

```
    private double y;
```

```
    private String renk;
```

```
    public static int toplamSekilSayisi;
```

```
    public void konumDegistir(double x, double y){
```

```
        renk = "Beyaz";
    }
```

```
    toplamSekilSayisi++;
```

```
}
```

GeometrikSekil

- x:double
- y:double
- renk:String="Beyaz"
+ toplamSekilSayisi: int

+ konumDegistir(double,double):void
+ alanHesapla() : double
+ toString() : String

+ : public
- : private
: protected
altı çizili: statik üye

GeometrikSekil sınıfının
gerçekleme seviyesi gösterimi

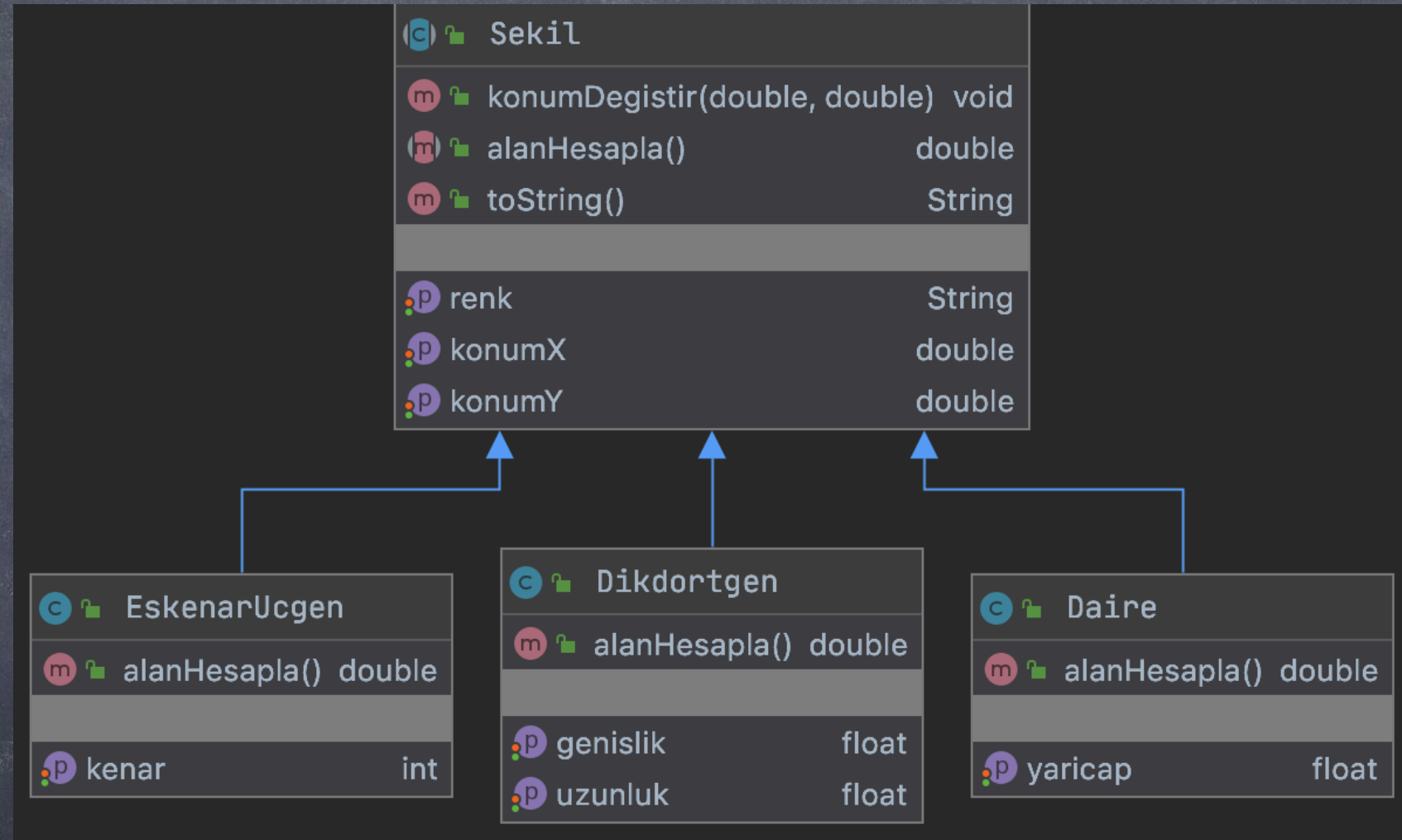
Veri Topluluklarıyla İşlemler

[https://github.com/celalceken/NesneYonelimliAnalizVeTasarimDersiUygulamalari/tree/master/Ders4/
VeriTopluluklariUzerindeIslemler](https://github.com/celalceken/NesneYonelimliAnalizVeTasarimDersiUygulamalari/tree/master/Ders4/VeriTopluluklariUzerindeIslemler)

Sınıf Diyagramı - Elementler arası bağıntılar

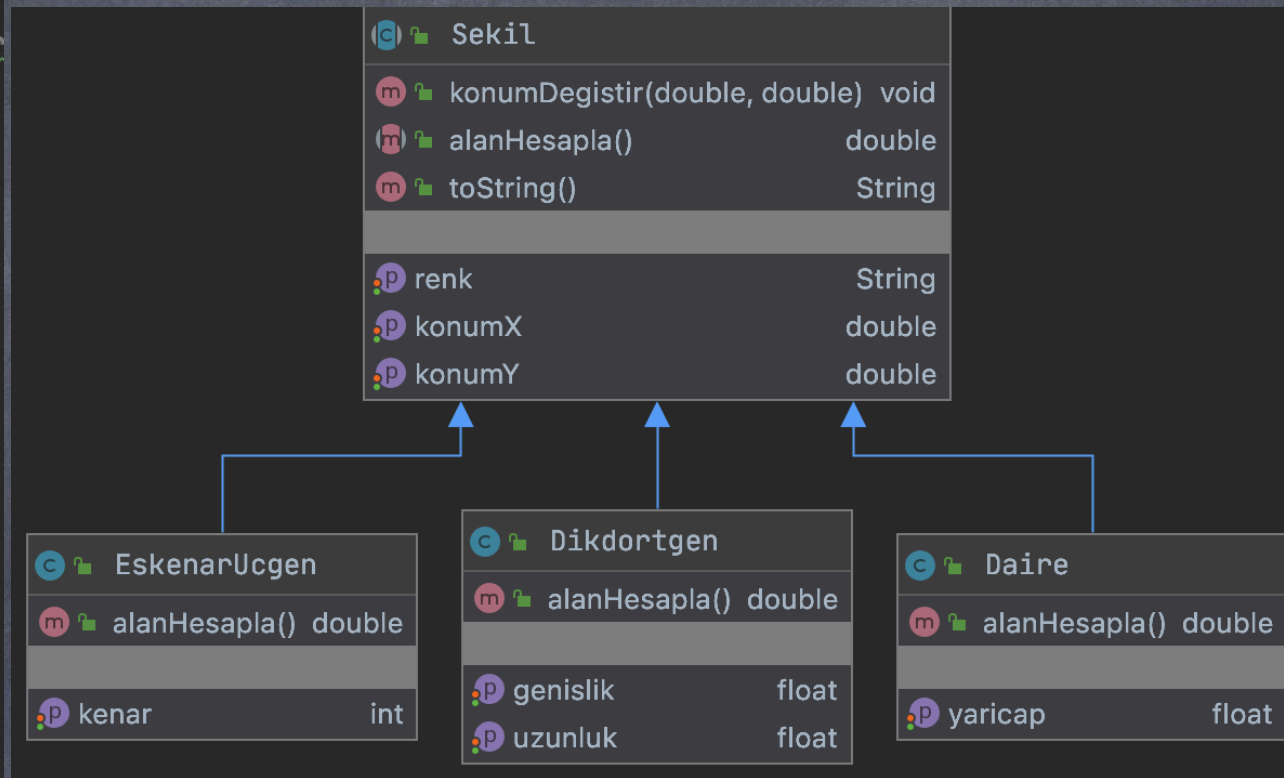
Kalıtım Bağıntısı

- * Benzer varlıklar arasında olmalıdır.
- * "is a", "is kind of"
- * Türetilmiş sınıf (Derived class) Temel sınıfın (base class) üyelerine kalıtım yoluyla sahip olur.
- * Kod tekrar kullanımı sağlanır



Sınıf Diyagramı - Elementler arası bağıntılar

```
abstract public class Sekil { //Soyut yöntem sadece soyut sınıflar  
    private double konumX;  
    private double konumY;  
    private String renk;  
  
    public Sekil(double konumX, double konumY, String renk) {  
        this.konumX = konumX;  
        this.konumY = konumY;  
        this.renk = renk;  
    }  
  
    public double getKonumX() { return konumX; }  
  
    public void setKonumX(double konumX) { this.konumX = konumX; }
```



```
public class Daire extends Sekil {  
  
    private float yaricap;  
  
    public Daire(double konumX, double konumY, String renk, float yaricap) {  
        super(konumX, konumY, renk);  
        this.yaricap = yaricap;  
    }  
  
    public float getYaricap() { return yaricap; }  
  
    public void setYaricap(float yaricap) { this.yaricap = yaricap; }
```

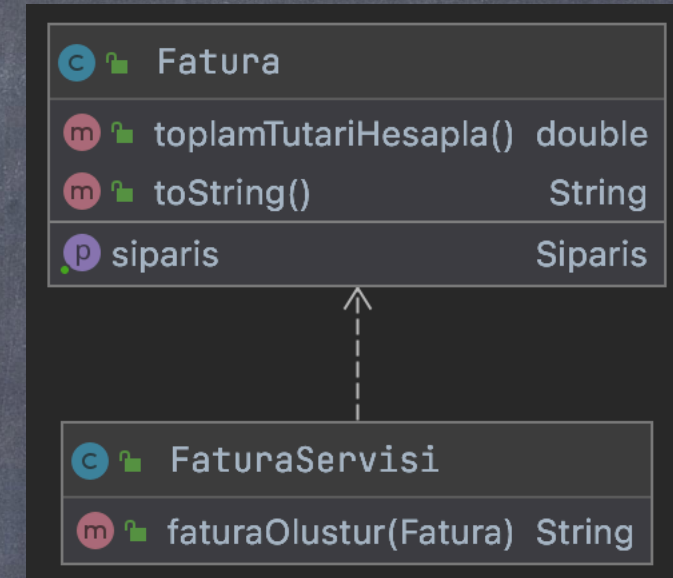
Sınıf Diyagramı - Elementler arası bağıntılar



"Dependency" Bağıntısı

- * Bir elementin (istemci- ClassA) diğer elementi (tedarikçi-ClassB) kullanmasını ifade eder.
- * "uses"
- * ClassA içerisindeki bir yöntem ClassB yi kullanır.

- * FaturaServisi sınıfında yer alan yöntem, sorumluluğunu yerine getirebilmek için Fatura elementini kullanır.



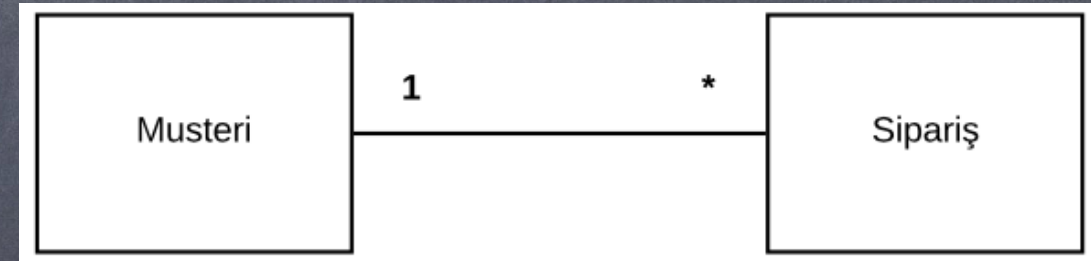
```
public class FaturaServisi {  
  
    public String faturaOlustur(Fatura fatura){  
  
        String faturaXML="<XML>"+fatura.toString();  
        return faturaXML;  
    }  
}
```



Sınıf Diyagramı - Elementler arası bağıntılar

"Association" Bağıntısı

- * Bir nesne diğer nesneyi üye (özellik) olarak içerir.



- * "has a"
- * "Bidirectional" (iki yönlü) ve unidirectional (tek yönlü) olabilir.
- * "Aggregation" ve "Composition" bağıntıları "Association" bağıntısının özel halleridir.
- * 1 Musteri nesnesi çok sayıda Siparis nesnesini (referansını) içerir.

Sınıf Diyagramı - Elementler arası bağıntılar

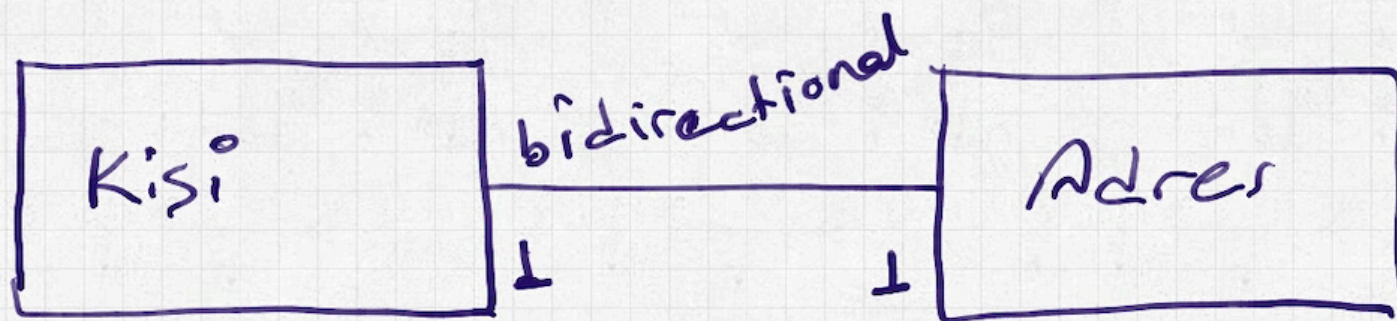
"Association" Bağlantısı



```
class Kisi
{
    private Adres adres;
    ...
}
```



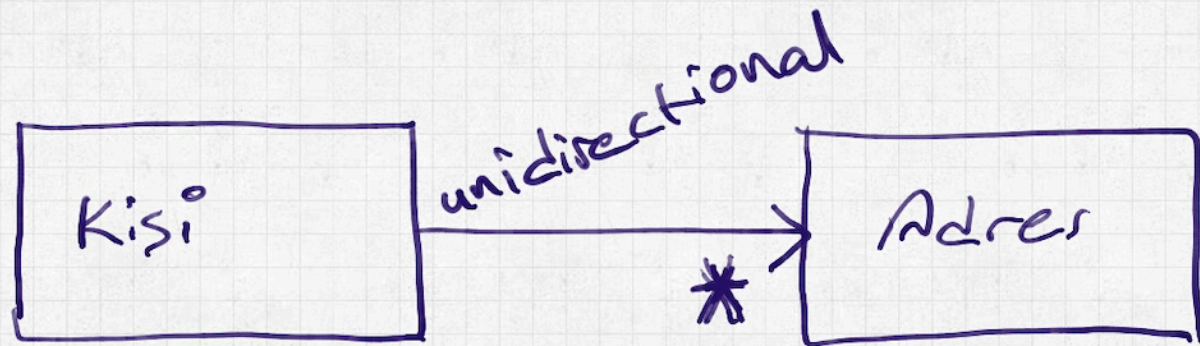
```
class Adres
{
    private Kisi kisi;
    ...
}
```



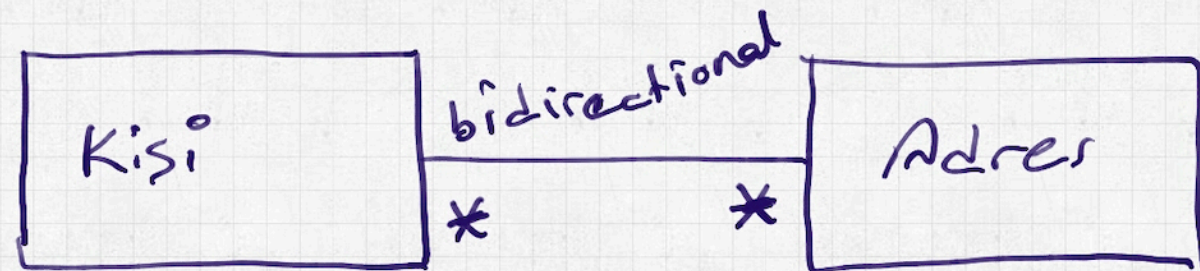
```
class Kisi
{
    private Adres adres;
    ...
}
class Adres
{
    private Kisi kisi;
    ...
}
```

Sınıf Diyagramı - Elementler arası bağlantılar

"Association" Bağlantısı



```
class Kisio
{
    private List<Adres> adresler;
}
```



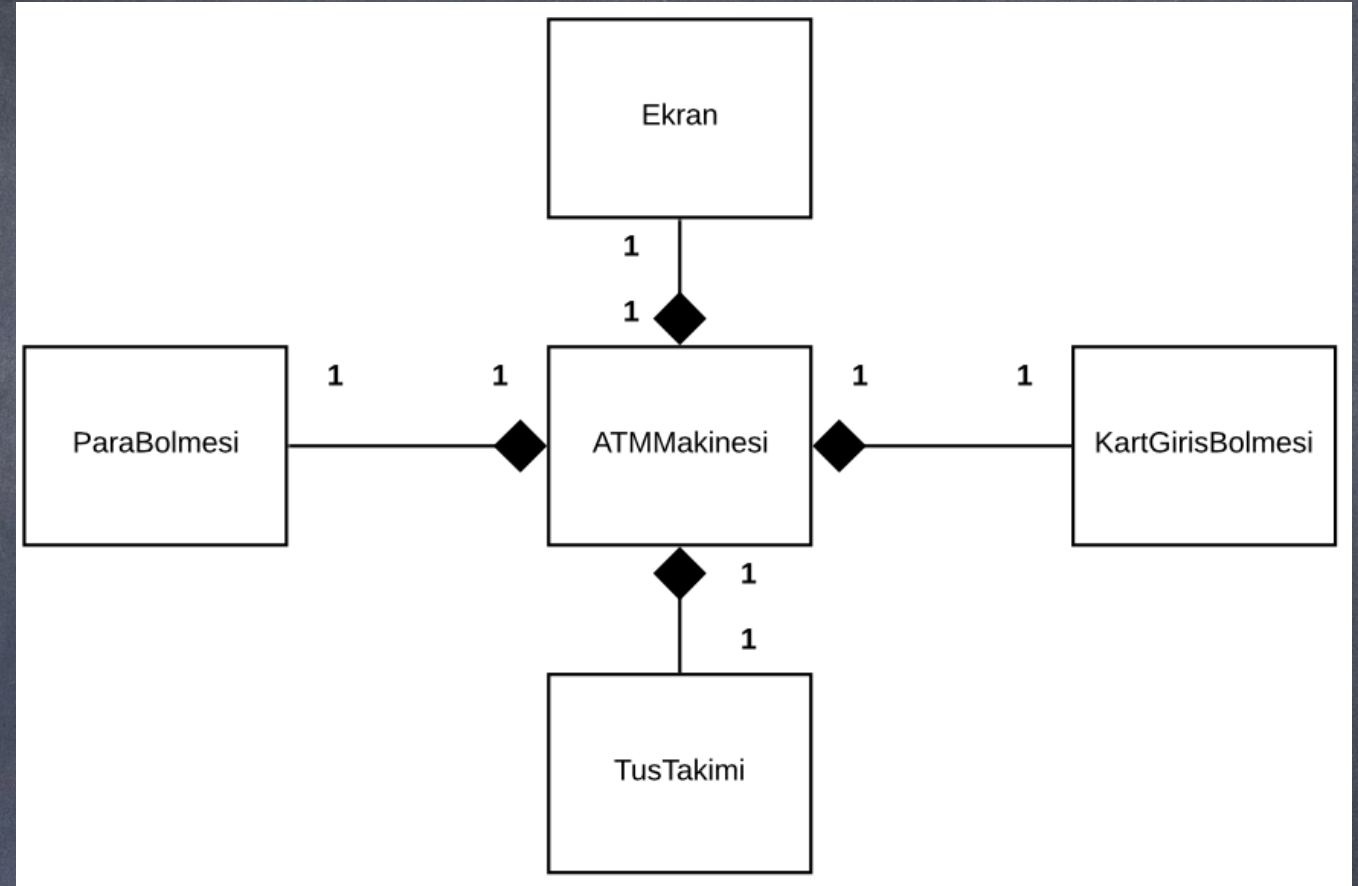
```
class Kisi
{
    private List<Adres> adresler;
}

class Adres
{
    private List<Kisi> kisiler;
}
```

Sınıf Diyagramı - Elementler arası bağıntılar

"Composition" Bağıntısı

- * Bütün parça bağıntısı (whole/part)
- * Bütün, işini yapabilmek için parçaya ihtiyaç duyar.
- * Parça nesneler bütün içerisinde yer alırlar. parçaları bütün oluşturur ve yok eder.
- * "is part of" (whole/part)
- * Her parça en çok 1 bütün içerisinde bulunur. Bir bütünün çok sayıda parça yer alabilir.



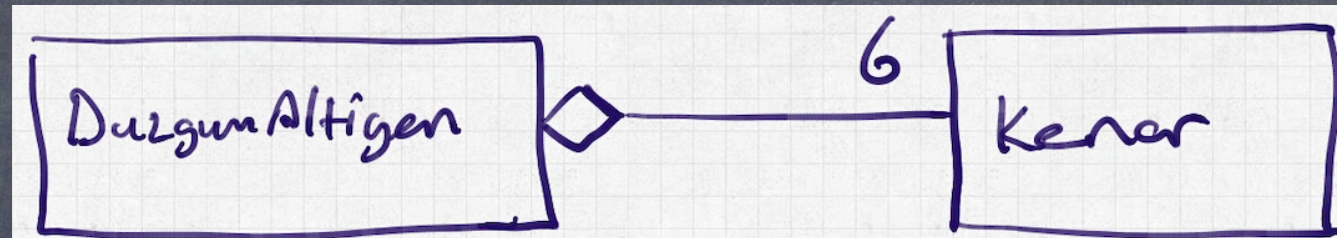
```
public class ATMMakinesi
{
    private Ekran ekran;
    private TusTakimi tusTakimi;
    private ParaBolmesi paraBolmesi;
    private KartGirisBolmesi kartBolmesi;

    public ATM() {
        ekran=new Ekran();
        tusTakimi=new TusTakimi();
        paraBolmesi=new ParaBolmesi();
        kartBolmesi=new KartGirisBolmesi();
    }
}
```

Sınıf Diyagramı - Elementler arası bağıntılar

"Aggregation" Bağıntısı

- * Bütün parça bağıntısı (whole/part)
- * Bütün, işini yapabilmek için parçaya ihtiyaç duyar.
- * Parça nesneler bütün içerisinde yer alırlar. parçalar başka bütün içerisinde de kullanılabilir. Bütün yok edildiğinde parçanın yok edilmesi gerekmebilir (başka bütün içerisinde kullanılıyor olabilir)
- * "consists of"

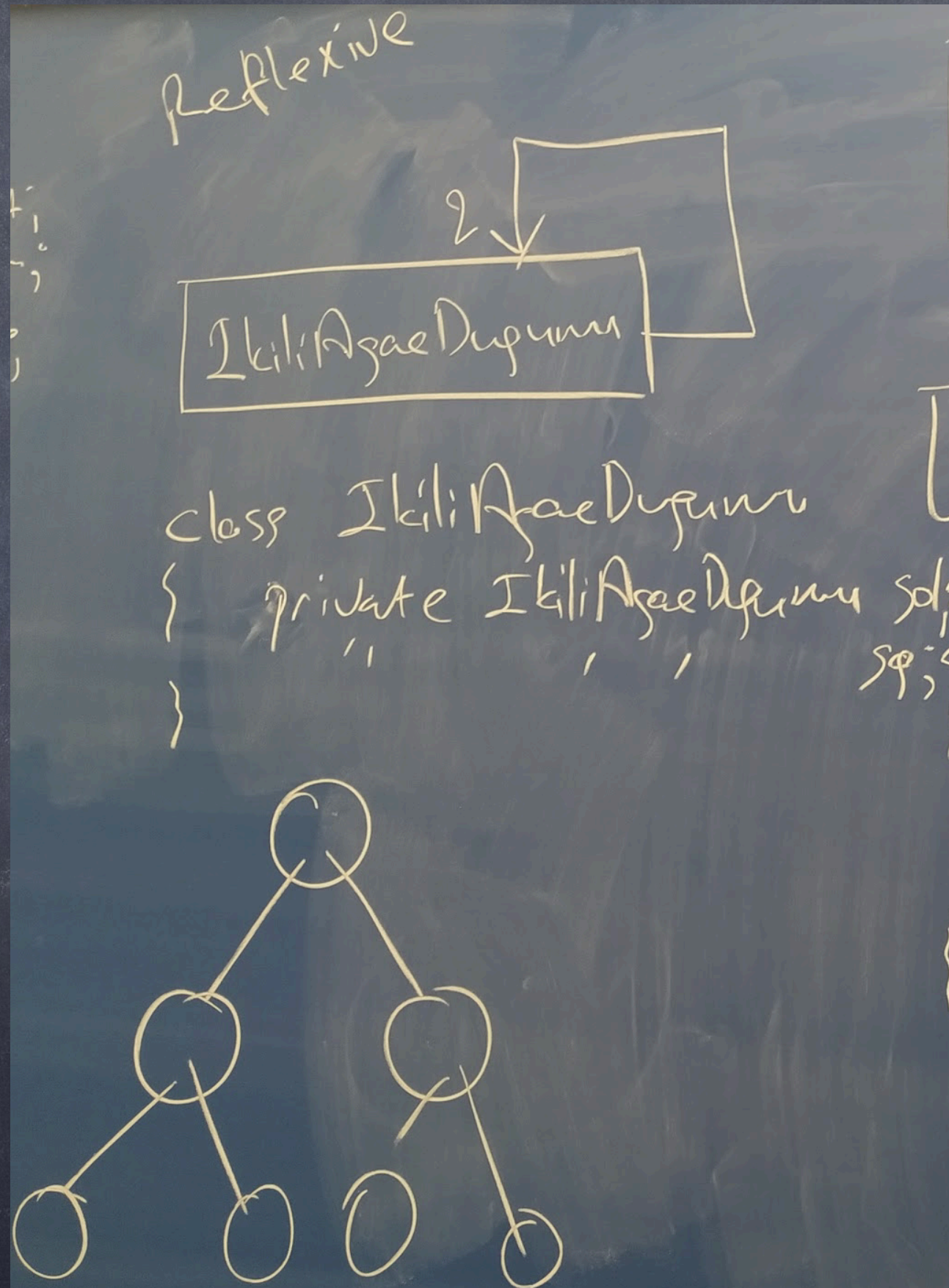


```
Class DuzgunAltigen {
```

```
    private Kenar kenar[] = new Kenar[6];
    ;
    ;
}
```

Sınıf Diyagramı - Elementler arası bağıntılar

* "Reflexive" Bağıntısı



Sınıf Diyagramı - Elementler arası bağıntılar

"Interface"

- * Bir sınıfın yapmasını istediklerimizi belirtmek için kullanırız ("is capable of"). Ne yapması gerektiği belirtilir, nasıl ile ilgilenilmez.

Arayüzler sayesinde:

- * gerçeklemeler(sınıflar) henüz ortada yok iken istemci kod içerisinde kullanılabilir
- * gerçekleştirme ve istemci modüller aynı anda farklı kodlayıcılar tarafından oluşturulabilir (takım çalışması)
- * yeni özellik eklendiğinde istemci kod değiştirilmez (sözleşme yapılır)
- * istemci kod, gerçekleştirme kısmındaki değişikliklerden etkilenmez (bağımlılığın zayıflatılması)
- * aynı istemci kod farklı modüller tarafından defalarca kullanılabilir (kod tekrar kullanımı-code reuse)

```
interface IVeritabaniSurucu {  
  
    public void baglan();  
    public void sorguCalistir();  
    public void baglantiSonlandir();  
}
```

I VeritabaniSurucu

- baglan() void
- sorguCalistir() void
- baglantiSonlandir() void

MySQLSurucu

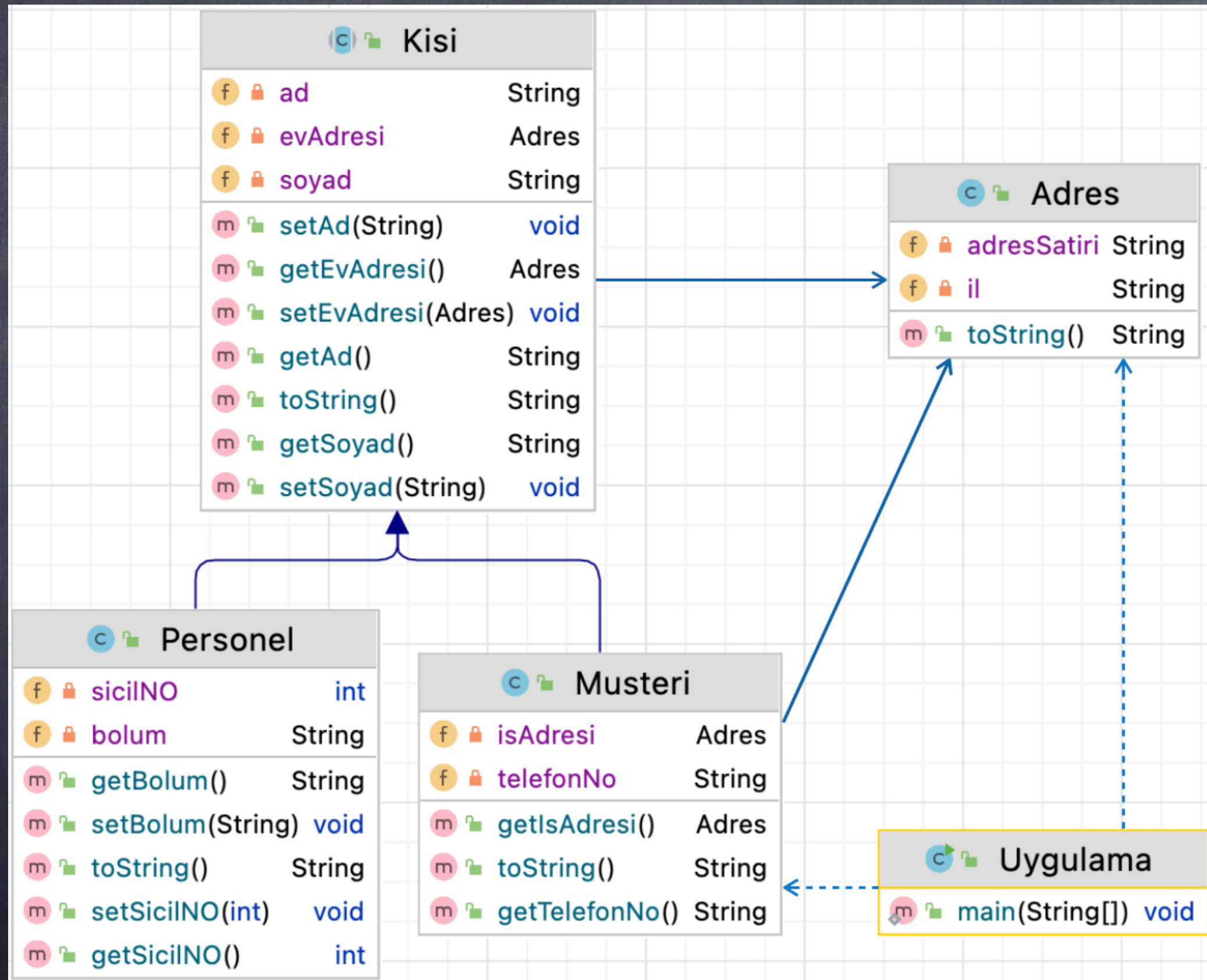
- baglantiSonlandir() void
- sorguCalistir() void
- baglan() void

PostgreSQLSurucu

- baglantiSonlandir() void
- sorguCalistir() void
- baglan() void

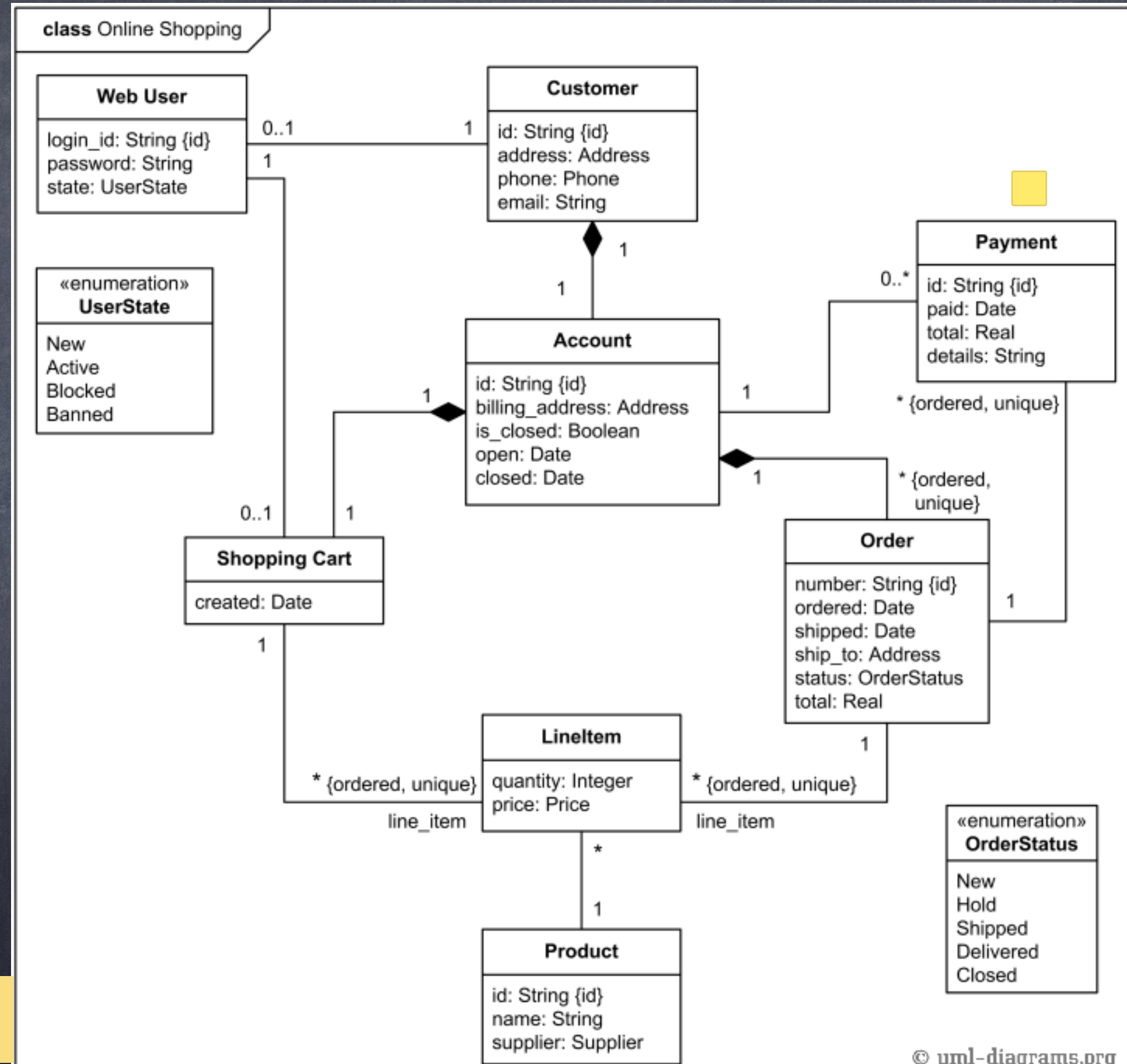
```
public class PostgreSQLSurucu implements IVeritabaniSurucu {  
  
    public void baglan() {  
        System.out.println("PostgreSQL veritabanına bağlanıyor...");  
    }  
  
    public void sorguCalistir() {  
        System.out.println("PostgreSQL sorgusu çalıştırılıyor...");  
    }  
  
    public void baglantiSonlandir() {  
        System.out.println("PostgreSQL veritabanı bağlantısı sonlandırılıyor...");  
    }  
}
```

Örnek Uygulama: Elementler arası bağıntılar

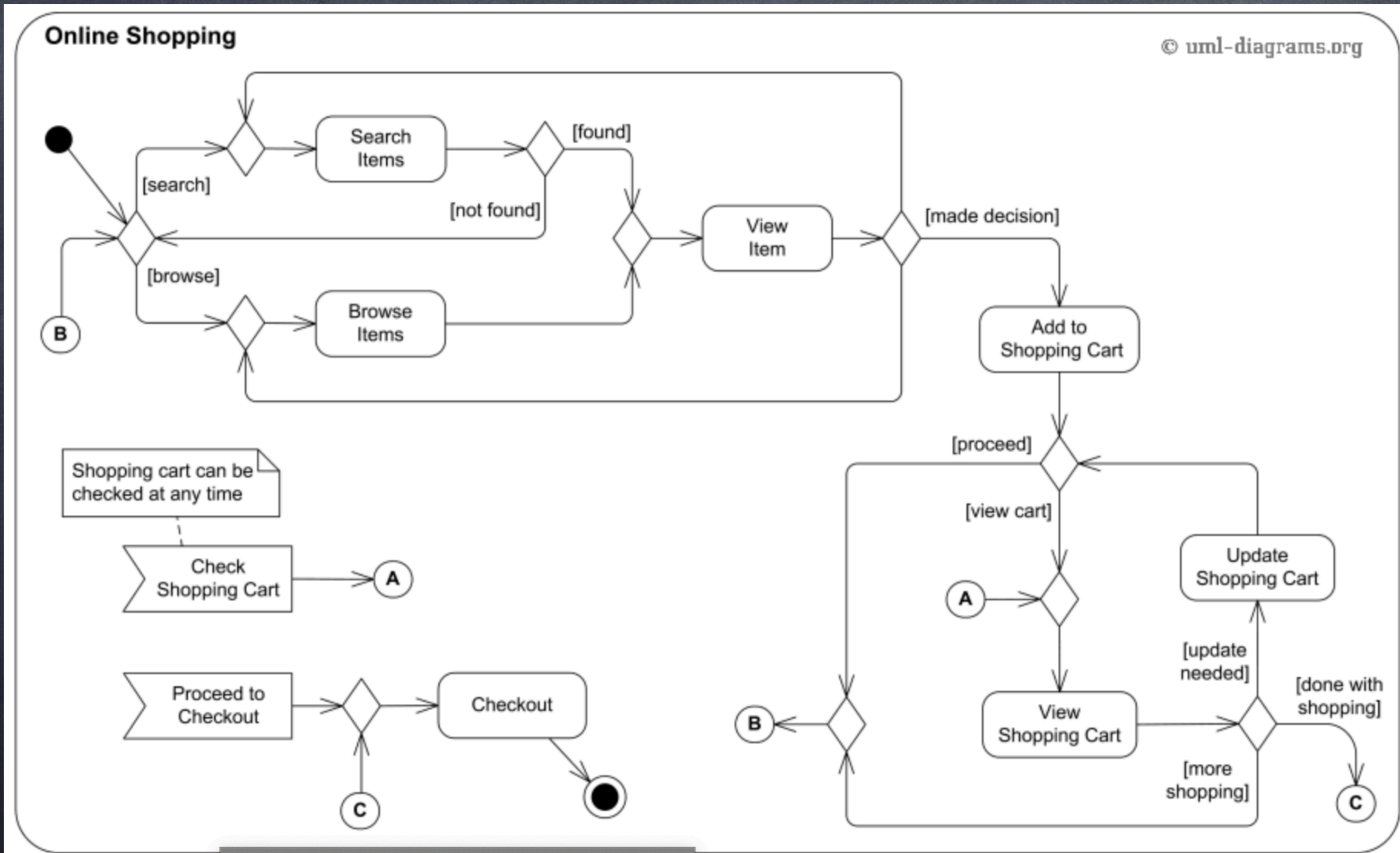


Örnek: Çevrimiçi Alış Veriş Sistemi Sınıf Şeması

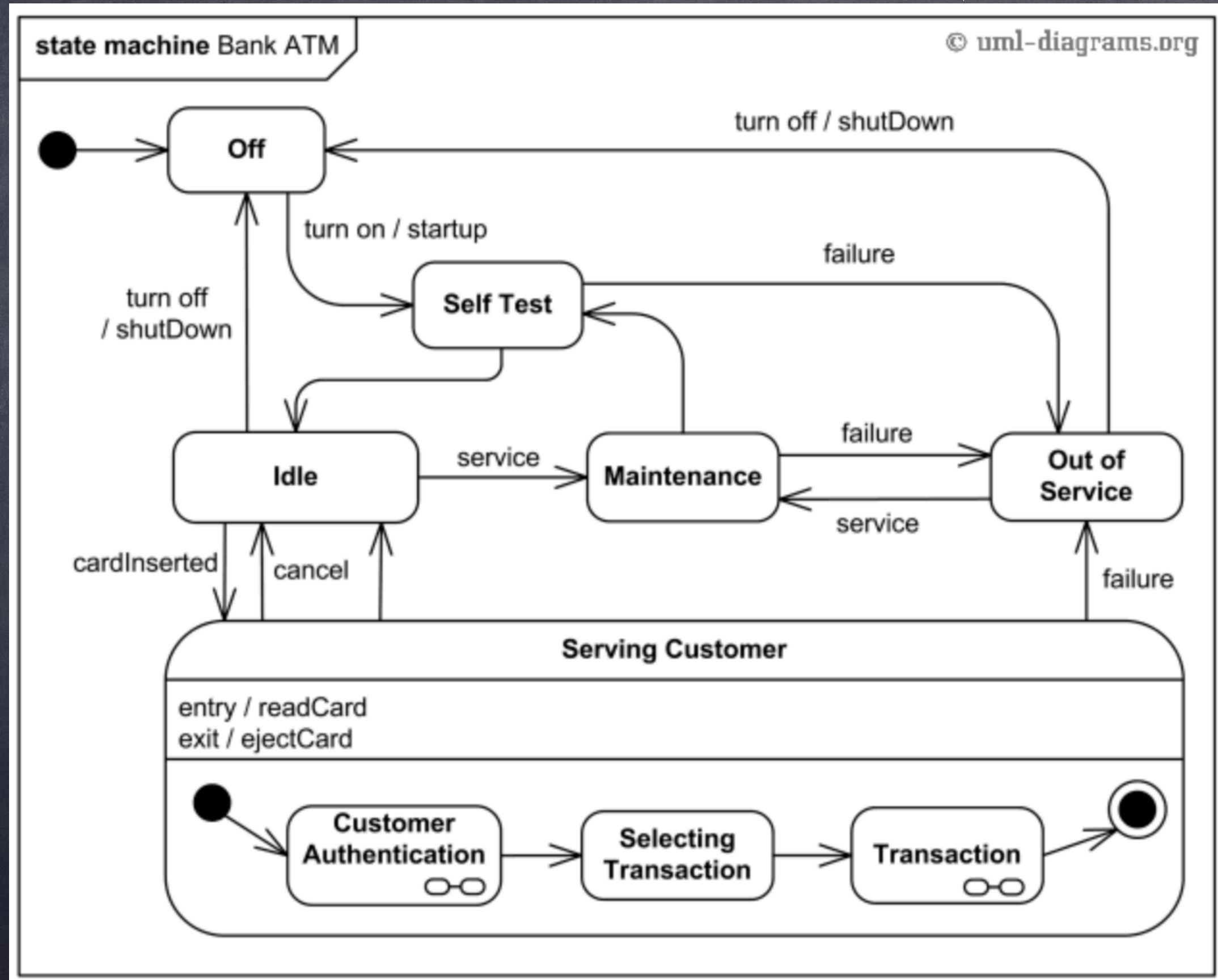
- * Her müşterinin eşsiz bir "id" si ve sadece bir hesabı vardır.
- * Hesaba ait bir alışveriş kartı (sepeti) (internet üzerinde alışveriş için) ve siparişler (klasik alışveriş, telefon, yüz yüze vb.) (0 ya da çok) vardır.
- * Sipariş ve alışveriş kartı alışveriş kalemlerine sahiptir.
- * Alışveriş kalemleri mutlaka bir ürünle ilgilidir. Ürünler çok sayıda alışveriş kalemleriyle ilgili olabilir.



Örnek: Çevrimiçi Alışveriş Sistemi Etkinlik Şeması



Örnek: ATM Durum Makinası Diyagramı



Örnek: E-Ticaret Uygulaması

Belirtilen gereksinimlere (iş kurallarına) göre nesneleri belirleyerek sınıf şemasını oluşturunuz.

Senaryo

Elektornik ticaret yazılımının ihtiyacı olan verilerin yönetimi için bir veritabanı geliştirilmesi isteniyor. İş kuralları verilen bu veritabanının VB diyagramını ve ilişkisel şemasını oluşturunuz.

İş Kuralları

- Bu veritabanında her müşteriye bir numara verilerek müşterinin TC Kimlik numarası, adı, soyadı, yaşadığı ili saklanması düşünülüyor.
- İllerin plaka numarası ve adı saklanır.
- Her siparişe bakan bir satış temsilcisi mevcuttur. Satış temsilcilerinin TCKimlikNo, ad ve soyad bilgileri mevcuttur.
- Ürünlerin (kişisel bilgisayar, telefon vb.) kodu, adı, fiyatı ve stok miktarlarının saklanması gerekmektedir.
- Ürünlerin kategorileri (bilgisayar, ev elektroniği, kozmetik vb.) mevcuttur.
- Müşterilerin ürün siparişleri saklanarak her bir siparişe bir fatura kesilmesi sağlanmalıdır.
- Her siparişin eşsiz bir sipariş numarası ve sipariş tarihi mevcuttur.
- Sipariş edilen bir ürünün sipariş adedi ve birim fiyatı (kişiye özel indirim v.s. nedeniyle ürün tablosundaki fiyattan farklılık gösterebilir) da kaydedilmelidir.
- Faturaların fatura numarası, tarih ve fatura adresi bilgileri saklanmalıdır. (Toplam fiyat hesaplanabilir ya da saklanabilir).
- Siparişler bir kargo firması tarafından iletilir. Kargo firmasının kodu, adı, adresi bilgileri yer alır. Her kargo firmasında siparişlerden sorumlu bir yetkili yer alır.
- Bir ürünün yalnızca bir kategorisi mevcuttur. Bir kategori çok sayıda ürünün kategorisi olabilir.
- Bir siparişte en az bir ürün bulunur. Ancak çok sayıda ürün de bulunabilir. Bir ürün çok sayıda siparişte yer alabilir.
- Bir müşteri çok sayıda sipariş verebilir. Bir sipariş yalnızca bir müşteri tarafından verilebilir.
- Bir siparişin yalnızca bir faturası olabilir. Bir fatura yalnızca bir siparişin faturası olabilir.
- Bir sipariş ile yalnızca bir satış temsilcisi ilgilenir. Bir satış temsilcisi çok sayıda sipariş ile ilgilenebilir.
- Bir müşteri yalnızca bir ilde yaşayabilir. Bir ilde çok sayıda müşteri yaşayabilir.
- Bir sipariş yalnızca bir kargo firması tarafından iletilir. Bir kargo firması çok sayıda sipariş iletebilir.